
Binary Image Classification on MNIST

Using KNN, Logistic Regression, and SVM

Technical Report

Submitted by

Abdel Rahman Khaled Gaber	ID: 23P0065
Ahmed Walid El Sayed	ID: 23P0038
Seif El Din Tamer	ID: 23P0240
Asaad Ramzy Asaad	ID: 23P0228

Git Hub Link: <https://github.com/seiftamerr/Binary-Image-Classification-on-MNIST-KNN-Logistic-Regression-SVM-.git>

Table of Contents

Table of Contents 2

1. Abstract 3

2. Introduction..... 3

3. Problem Definition..... 4

4. Dataset and Preprocessing 4

 4.1 Dataset..... 4

 4.2 Preprocessing Pipeline 4

5. Model Implementation..... 5

 5.1 K-Nearest Neighbors (KNN) 5

 5.2 Logistic Regression..... 6

 5.3 Linear SVM 7

6. Experimental Setup 7

7. Evaluation Metrics 8

8. Results..... 8

9. Discussion 9

 9.1 Comparative Performance 9

 9.2 Bias-Variance Tradeoff..... 9

 9.3 Computational Cost 9

10. Conclusion 10

1. Abstract

This report presents a comparative study of three classical supervised machine learning algorithms — K-Nearest Neighbors (KNN), Logistic Regression, and a linear Support Vector Machine (SVM) — applied to binary image classification using the MNIST handwritten digit dataset, restricted to classes 0 and 1. Each model is implemented from scratch using NumPy, trained on a shared and rigorously preprocessed dataset, and evaluated using accuracy, precision, recall, and F1-score. All three classifiers achieve test accuracies exceeding 99.3%, with Logistic Regression achieving the best overall result at 99.76%. The report covers mathematical formulations, optimization procedures, hyperparameter choices, code snippets, and a critical discussion of each model's tradeoffs in terms of accuracy, generalization, and computational cost.

2. Introduction

Image classification is one of the most studied problems in machine learning. The MNIST dataset — 70,000 grayscale 28×28 images of handwritten digits — has served as a benchmark for evaluating learning algorithms since its introduction by LeCun et al. (1998). This project restricts the multi-class problem to binary classification between digit 0 (a closed oval) and digit 1 (a vertical stroke), two visually distinct classes that allow us to study the fundamental behavior of three learning paradigms without multi-class complexity.

We implement: (1) **KNN** — a non-parametric instance-based learner with no training phase; (2) **Logistic Regression** — a probabilistic linear classifier trained by maximizing log-likelihood via gradient descent; and (3) **Linear SVM** — a maximum-margin classifier trained with a subgradient method. All models share identical preprocessing and evaluation pipelines, ensuring a fair comparison. The goal of this report is to document the theoretical foundations, implementation details, and practical tradeoffs of each approach.

3. Problem Definition

Let the input space be $X = \mathbb{R}^{784}$, where each sample $\mathbf{x} \in \mathbb{R}^{784}$ is a flattened 28×28 grayscale image. The label space is $Y = \{0, 1\}$. The task is to learn a function $f: \mathbb{R}^{784} \rightarrow \{0, 1\}$ minimizing the expected misclassification risk:

$$R(f) = E[1(y \neq f(x))] \quad \text{approximated by} \quad \hat{R}(f) = (1/N) \sum_i 1(y_i \neq f(\mathbf{x}_i))$$

Since the digit classes 0 and 1 are visually highly distinct, the Bayes error rate is near zero — making this a near-linearly separable binary problem in the standardized pixel space. Each classifier constructs f differently: KNN via local majority voting, Logistic Regression by estimating $P(y=1|\mathbf{x})$, and SVM by locating the maximum-margin separating hyperplane.

4. Dataset and Preprocessing

4.1 Dataset

Filtering MNIST to classes $\{0,1\}$ yields approximately 12,665 training samples and 2,115 test samples. The class ratio is ~1.14:1 (digit 1 slightly more frequent), small enough to require no resampling.

4.2 Preprocessing Pipeline

Step 1 — Filtering: Retain only samples with label $\in \{0, 1\}$ using boolean masks on train and test label arrays.

Step 2 — Normalization to [0,1]: Divide all pixel values by 255.0. This maps integer values in $[0, 255]$ to $[0.0, 1.0]$, preventing large-magnitude inputs from destabilizing gradient-based updates and equalizing feature scales for distance-based methods.

Step 3 — Flattening: Reshape each 28×28 image into a 784-dimensional vector, the required format for all three classifiers.

Step 4 — Stratified Train/Validation Split: The filtered training data is split 85%/15% using stratified sampling (seed = 42), giving ~10,741 train and ~1,896 validation samples. The official test split is held out until final evaluation.

Step 5 — Standardization: Zero-mean, unit-variance normalization using training set statistics only:

$$\tilde{x} = (x - \mu) / \sigma \quad \text{where } \mu, \sigma \text{ are computed on the training set}$$

Pixels with $\sigma = 0$ are assigned $\sigma = 1$. The same μ and σ are applied to validation and test sets to prevent data leakage.

Code — Data Preprocessing Pipeline:

```
# Normalize to [0, 1]
X_train_full = X_train_full.astype(np.float32) / 255.0
X_test        = X_test.astype(np.float32) / 255.0

# Flatten 28x28 → 784
X_train_full = X_train_full.reshape(X_train_full.shape[0], -1)
X_test        = X_test.reshape(X_test.shape[0], -1)

# Stratified 85/15 train/val split
X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full,
    test_size=0.15, random_state=42, stratify=y_train_full)

# Standardize using training statistics only
mean = X_train.mean(axis=0)
std   = X_train.std(axis=0)
std[std == 0] = 1
X_train = (X_train - mean) / std
X_val    = (X_val - mean) / std
X_test   = (X_test - mean) / std
```

5. Model Implementation

5.1 K-Nearest Neighbors (KNN)

KNN is a non-parametric lazy learner: the entire training set is stored and no parameters are learned. At inference, the k nearest training points to query $\mathbf{x}$ are found by Euclidean distance, and the majority label is returned:

$$f(x) = \operatorname{argmax}_c \{c \in \{0, 1\} \mid \sum_i \mathbb{1}[y_i = c] \geq \frac{k}{2}\}$$

Distances are computed efficiently using the identity $\|\mathbf{x} - \mathbf{x}_i\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{x}_i\|^2 - 2\mathbf{x}^T \mathbf{x}_i$, with squared norms precomputed at fit time. The k nearest neighbors are retrieved in $O(N)$ via **np.argpartition**, avoiding a full $O(N \log N)$ sort.

Code — KNN Core Methods:

```

def fit(self, X, y):
    self.X_train = np.asarray(X, dtype=np.float32)
    self.y_train = np.asarray(y, dtype=np.int64)
    # Precompute squared norms for fast distance calculation
    self.X_train_norm = np.sum(self.X_train ** 2, axis=1)

def _euclidean_distances(self, x):
    # ||a-b||^2 = ||a||^2 + ||b||^2 - 2a.b
    x_norm = np.sum(x ** 2)
    return self.X_train_norm + x_norm - 2 * np.dot(self.X_train, x)

def _predict_single(self, x):
    distances = self._euclidean_distances(x)
    k_indices = np.argpartition(distances, self.k)[:self.k] # O(N)
    k_labels = self.y_train[k_indices]
    return np.bincount(k_labels).argmax()

```

5.2 Logistic Regression

Logistic Regression models the posterior class probability as a sigmoid of a linear function:

$$P(y=1 \mid x) = \sigma(w^T x + b) \quad \sigma(z) = 1 / (1 + e^{(-z)})$$

Prediction: $f(\mathbf{x}) = 1$ if $P(y=1|\mathbf{x}) \geq 0.5$, else 0. Parameters $\mathbf{w} \in \mathbb{R}^{784}$ and b are learned by minimizing the binary cross-entropy loss with optional L2 regularization over 1000 epochs of batch gradient descent ($\eta = 0.1$).

$$J(w, b) = -(1/m) \sum_i [y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)] + (\lambda/2m) \|\mathbf{w}\|^2$$

Code — Logistic Regression Training Loop:

```

def fit(self, X, y):
    m, n = X.shape
    self.weights = np.zeros(n)
    self.bias = 0.0
    for epoch in range(self.epochs):
        # Forward pass
        z = np.dot(X, self.weights) + self.bias
        y_pred = self._sigmoid(z)
        # Gradients (vectorized over all samples)
        dw = (np.dot(X.T, (y_pred - y)) / m)
            + (self.lambda_ / m) * self.weights
        db = np.sum(y_pred - y) / m
        # Parameter update
        self.weights -= self.learning_rate * dw
        self.bias -= self.learning_rate * db

```

5.3 Linear SVM

The SVM seeks the maximum-margin hyperplane with decision function $f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} - b)$. Training maps labels to $\{-1, +1\}$ and minimizes the regularized hinge loss:

$$J(\mathbf{w}, b) = \lambda \|\mathbf{w}\|^2 + (1/m) \sum_i \max(0, 1 - y_i (\mathbf{w}^T \mathbf{x}_i - b))$$

A subgradient update is applied per sample at each iteration. If the margin constraint $y_i(\mathbf{w}^T \mathbf{x}_i - b) \geq 1$ holds, only the regularization term updates $\mathbf{w}$; otherwise the hinge correction is added.

Code — SVM Subgradient Update:

```
def fit(self, X, y):
    y_ = np.where(y <= 0, -1, 1)  # map labels to {-1, +1}
    self.w = np.zeros(X.shape[1], dtype=np.float32)
    self.b = 0.0
    for _ in range(self.n_iters):
        for idx, x_i in enumerate(X):
            margin = y_[idx] * (np.dot(x_i, self.w) - self.b)
            if margin >= 1:
                # constraint satisfied
                dw = 2 * self.lambda_param * self.w
                db = 0
            else:
                # margin violation
                dw = 2 * self.lambda_param * self.w - y_[idx] * x_i
                db = y_[idx]
            self.w -= self.lr * dw
            self.b -= self.lr * db
```

6. Experimental Setup

All models share the same preprocessing pipeline and train/validation/test split. Hyperparameters were selected by monitoring validation accuracy. Final configurations:

Model	Hyperparameter	Value
KNN	k (neighbors)	3
Logistic Regression	Learning rate / Epochs / λ	0.1 / 1000 / 0.0
Linear SVM	Learning rate / Iterations / λ	0.001 / 1000 / 0.01

7. Evaluation Metrics

Model performance is assessed using four complementary metrics (positive class = digit 1):

Accuracy: $(TP + TN) / (TP + TN + FP + FN)$ — overall fraction of correct predictions.

Precision: $TP / (TP + FP)$ — fraction of positive predictions that are truly positive.

Recall: $TP / (TP + FN)$ — fraction of actual positives correctly identified.

F1-Score: $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$ — harmonic mean balancing both.

The **Confusion Matrix** provides a complete picture: diagonal entries count correct predictions, off-diagonal entries reveal which class is systematically misclassified. For our near-perfectly separable task, misclassifications are extremely rare and roughly symmetric across all three models.

8. Results

The following table summarizes performance across all splits and metrics:

Model	Train Acc.	Val. Acc.	Test Acc.	Precision	F1-Score
KNN (k=3)	99.82%	99.47%	99.39%	0.9952	0.9941
Logistic Regression	99.91%	99.79%	99.76%	0.9980	0.9977
Linear SVM	99.87%	99.74%	99.67%	0.9971	0.9968

All three classifiers perform exceptionally well. The train-to-test accuracy gap is under 0.5% for every model, indicating negligible overfitting. Logistic Regression achieves the highest test accuracy (99.76%), followed by SVM (99.67%) and KNN (99.39%). Confusion matrices confirm that misclassifications are very rare and roughly symmetric — no model exhibits systematic bias toward either class.

9. Discussion

9.1 Comparative Performance

Logistic Regression marginally outperforms the SVM because its probabilistic objective (log-likelihood maximization) yields a decision boundary calibrated to the full data distribution, not just the support vectors near the margin. KNN's slightly lower score reflects the curse of dimensionality: in 784 dimensions, the nearest-neighbor structure becomes noisier, and a 3-vote majority is more susceptible to label noise than a global linear separator.

9.2 Bias-Variance Tradeoff

KNN ($k=3$) is a low-bias, higher-variance estimator — flexible enough to model non-linear boundaries but sensitive to local noise. The linear models (LR and SVM) are high-bias but low-variance: they impose a global linear boundary. Since the 0/1 task is near-linearly separable, the linear bias does not hurt accuracy, and L2 regularization further suppresses variance in both models. A larger k for KNN or stronger regularization for the linear models would shift each model further toward the high-bias end of the tradeoff.

9.3 Computational Cost

KNN requires $O(N \cdot d) \approx 8.4\text{M}$ operations per prediction ($N = 10,741$, $d = 784$). The `argpartition` optimization reduces the constant factor but not the asymptotic cost. Logistic Regression trains with vectorized batch gradient descent — $O(\text{epochs} \cdot N \cdot d)$ for training, but only $O(d)$ per prediction, making inference nearly instantaneous. The SVM's sample-wise subgradient loop is not vectorized, resulting in the highest wall-clock training time despite the same asymptotic complexity as Logistic Regression. A dual-form or mini-batch SVM implementation would be significantly faster in practice.

10. Conclusion

This project implemented and evaluated three classical supervised learning algorithms — K-Nearest Neighbors, Logistic Regression, and a linear SVM — on a binary MNIST classification task (digits 0 and 1). All models were built from scratch using NumPy and trained on an identical, rigorously preprocessed pipeline. The central findings are summarized below.

All three classifiers achieve test accuracies exceeding 99.3%, confirming that the 0/1 digit separation is nearly linearly separable in the standardized 784-dimensional pixel space. **Logistic Regression** emerges as the best overall model: highest test accuracy (99.76%), fastest inference (single matrix-vector product), and a principled probabilistic framework that generalizes robustly. **Linear SVM** is a close second (99.67%) with a geometrically motivated objective, but its sample-wise training loop makes it slower to converge. **KNN** (99.39%) is the simplest conceptually and requires no training, but its inference time scales linearly with training set size — a serious limitation for large-scale deployment.

In terms of bias-variance characteristics, the linear models maintain low variance through their global decision boundaries and regularization, while KNN trades lower bias for higher variance through local, data-driven decisions. On this near-linearly separable task, the linear bias causes no accuracy penalty; on more complex or overlapping distributions, KNN's flexibility could be advantageous.

These results reinforce a key lesson in machine learning: algorithm selection must account for more than raw accuracy. Factors such as inference latency, training time, interpretability, and scalability are equally important in practical settings. While deep learning approaches such as Convolutional Neural Networks would further reduce the already-tiny error rate, the classical algorithms studied here remain highly competitive on well-defined binary tasks and offer significantly better interpretability and lower computational overhead.

Future directions include applying dimensionality reduction (e.g., PCA) before training to assess whether a compressed representation improves KNN's performance in high-dimensional space, extending the comparison to the full 10-class MNIST problem, and benchmarking against a simple multilayer perceptron to quantify the marginal gain from non-linear function approximation.